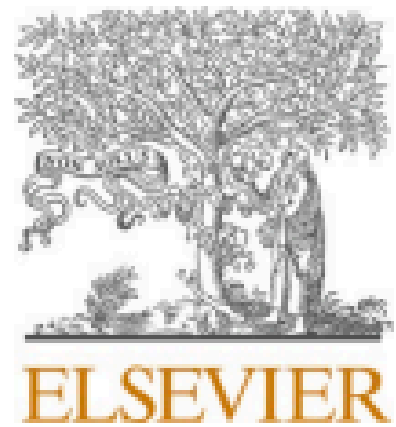
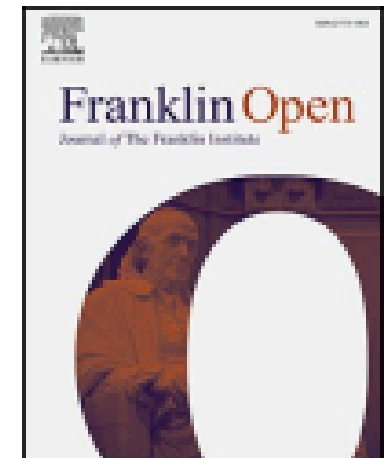


Franklin Open 12 (2025) 100322



Contents lists available at ScienceDirect

Franklin Open

journal homepage: www.elsevier.com/locate/fraope

Architecture review: Two-stage and one-stage object detection

Samiyaa Yaseen Mohammed 

Mustansiriyah University, College of Engineering, Department of Highway and Transportation Engineering, Baghdad, Iraq

삼석사와 아이들

어지유 이문용 김재현 김현진 박재익

Two-stage

목차

1. 논문 간단 소개
2. One-stage vs Two-stage
3. R-CNN 계열 진화
4. R-CNN & Fast R-CNN 아키텍처
5. Faster R-CNN & Mask R-CNN 아키텍처
6. PennFudanPed & 구현 설정
7. 결과 & 분석

제목: Architecture review: Two-stage and One-stage Object Detection

- 저자/게재: Samiyaa Yaseen Mohammed, Franklin Open (2025)
- 핵심 주제: 객체 검출의 두 축 Two-stage(R-CNN 계열) vs One-stage(SSD/RetinaNet/YOLO)의 아키텍처·진화·성능 트레이드오프를 체계적으로 비교 정리

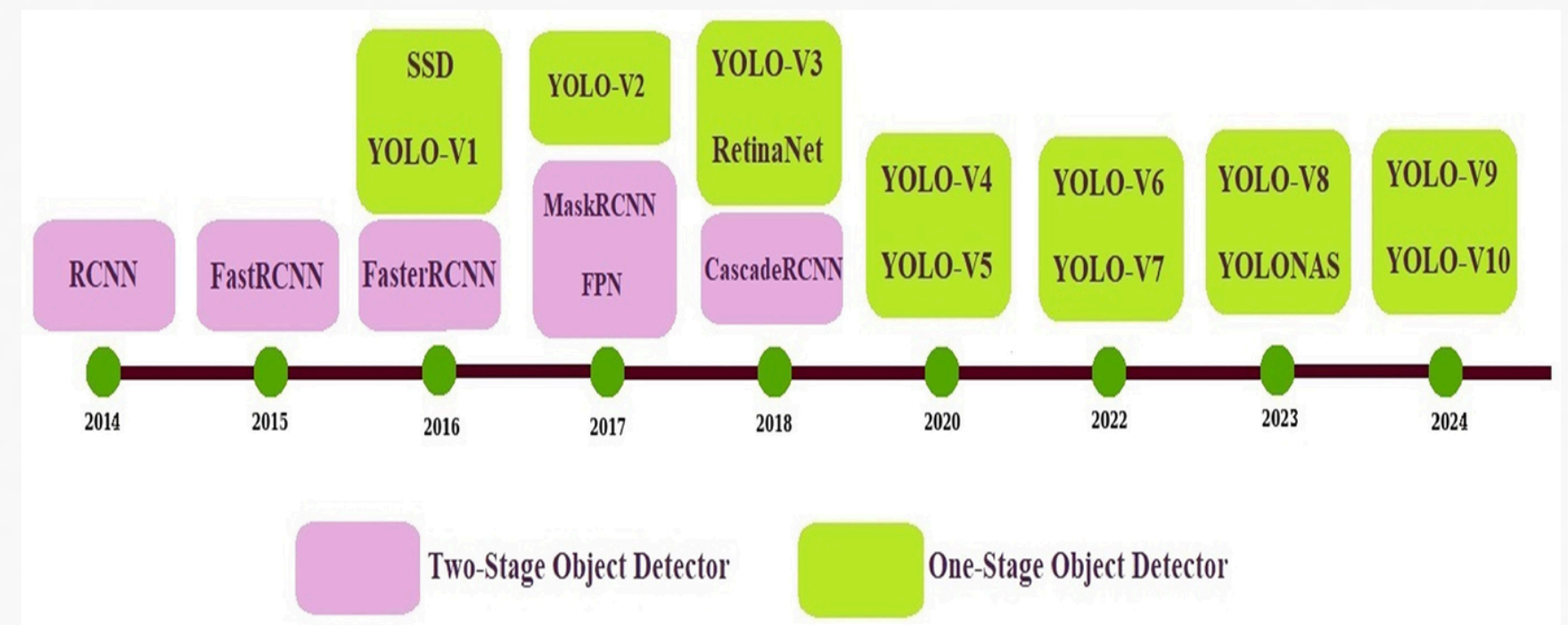
One-stage vs Two-stage

Two-stage 객체의 위치 및 분류를 각각 진행하는 객체 탐지

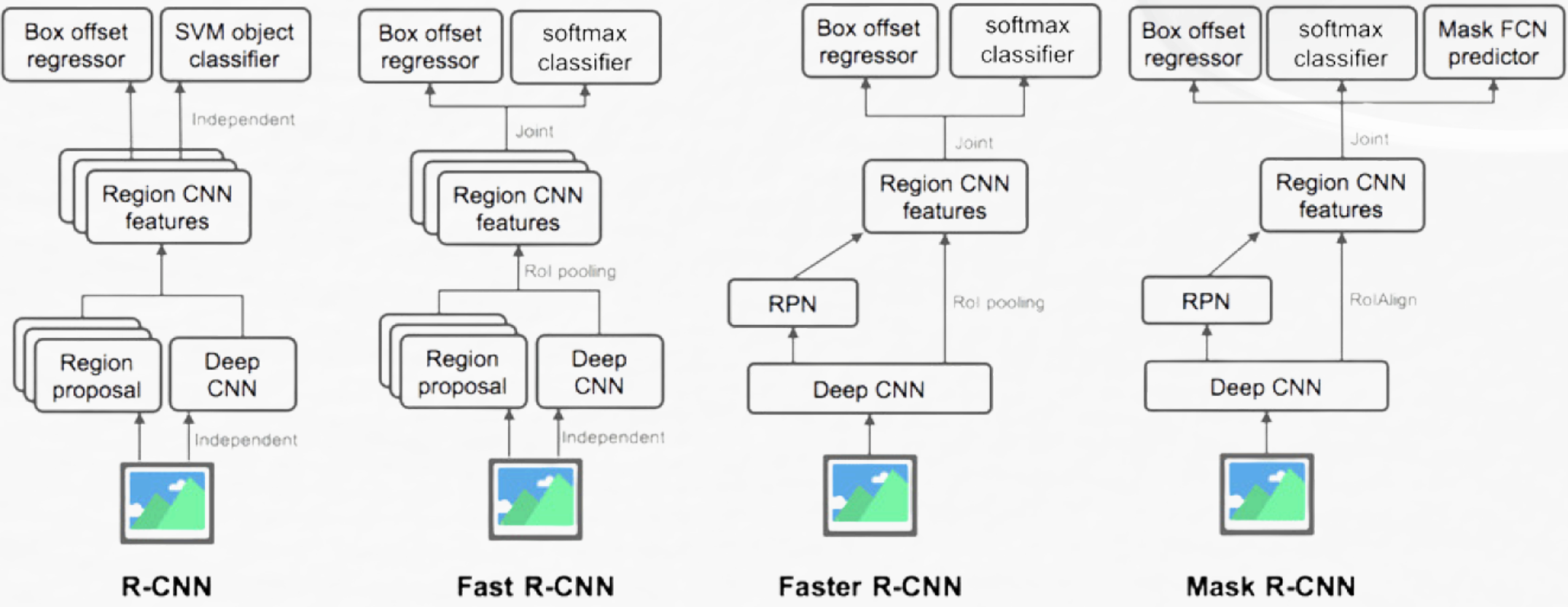
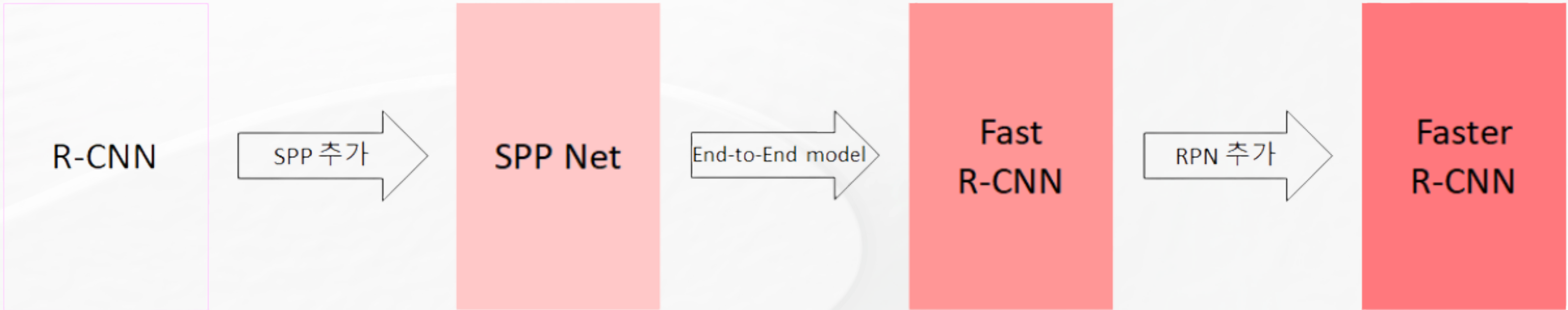
- 흐름: Region Proposal(후보영역) → RoI 정렬(Align) → 분류·박스 정제(필요 시 마스크)
- 주요 특징: 높은 정밀도, 작은/겹친 물체 강함, 상대적으로 느린 속도
- 대표: R-CNN, Fast/Faster R-CNN, Mask R-CNN, Cascade R-CNN
- 활용: 의료/제조 검사, 인스턴스 분할, 정밀 컷아웃(의류 마스크)

One-stage 객체의 위치 및 분류를 한 단계로 진행하는 객체 탐지

- 흐름: 다중 스케일 특성맵에서 직접 cls+box 회귀(Anchor/Anchor-free)
- 주요 특징: 실시간에 가까운 빠른 속도, 경량/배포 용이, 작은 물체는 상대적 약점(최근 개선 중)
- 대표: YOLO 계열, SSD, RetinaNet(학습: Focal Loss)
- 활용: 실시간 서비스, 엣지/모바일, 대량 스트림 모니터링



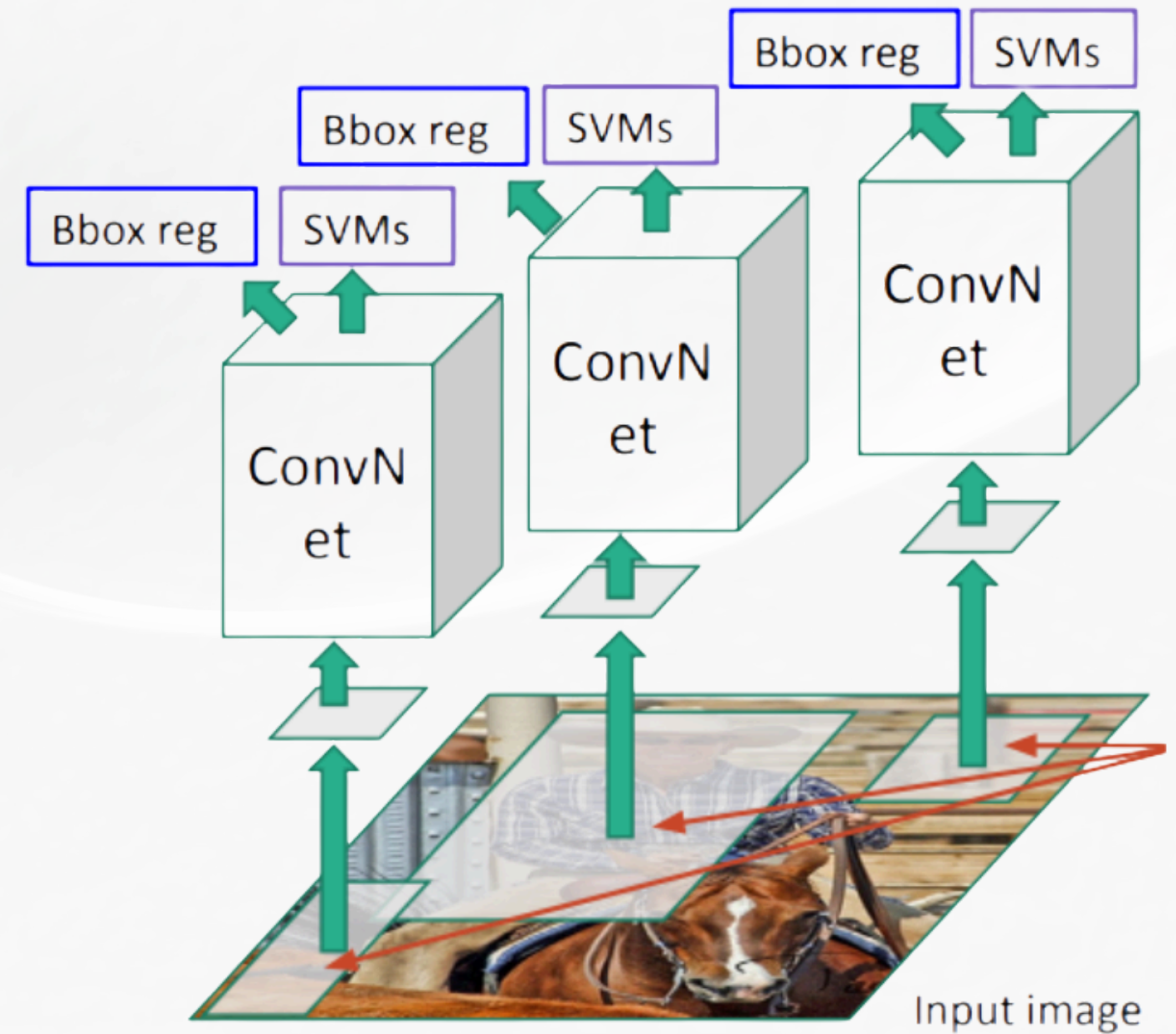
R-CNN 계열 진화



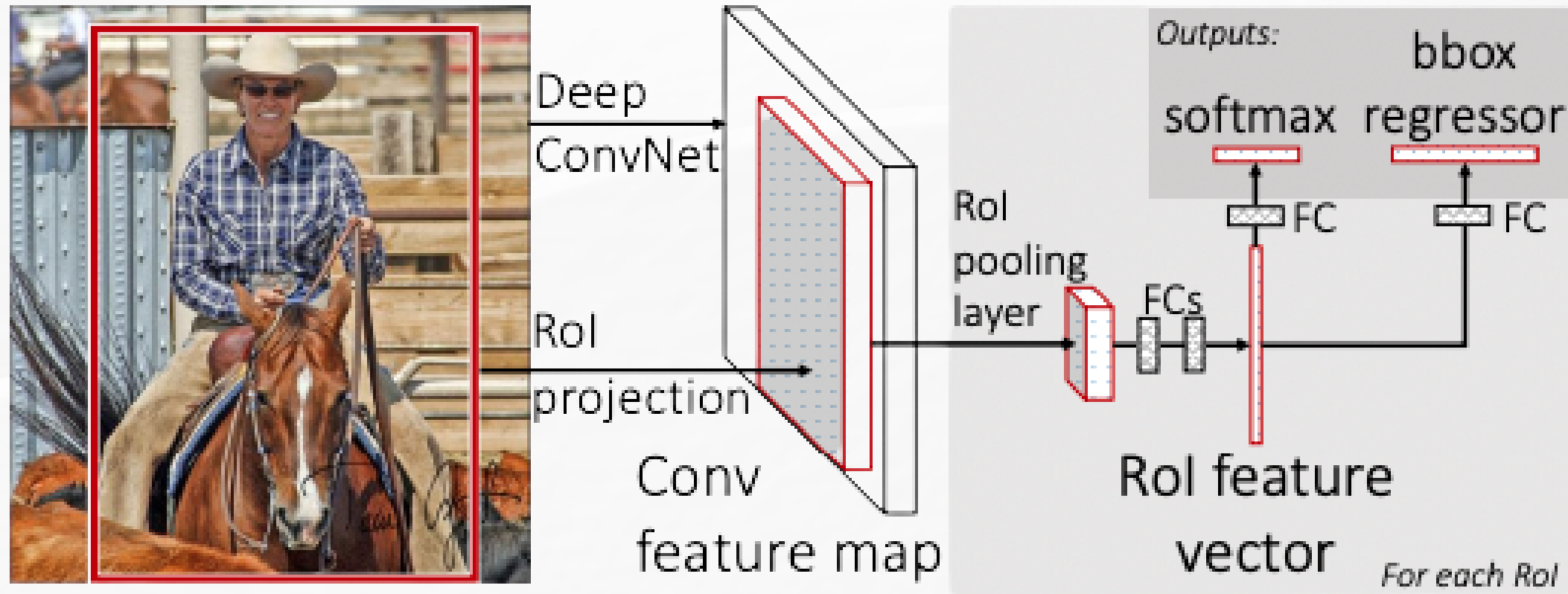
R-CNN 아키텍처

전체 파이프라인

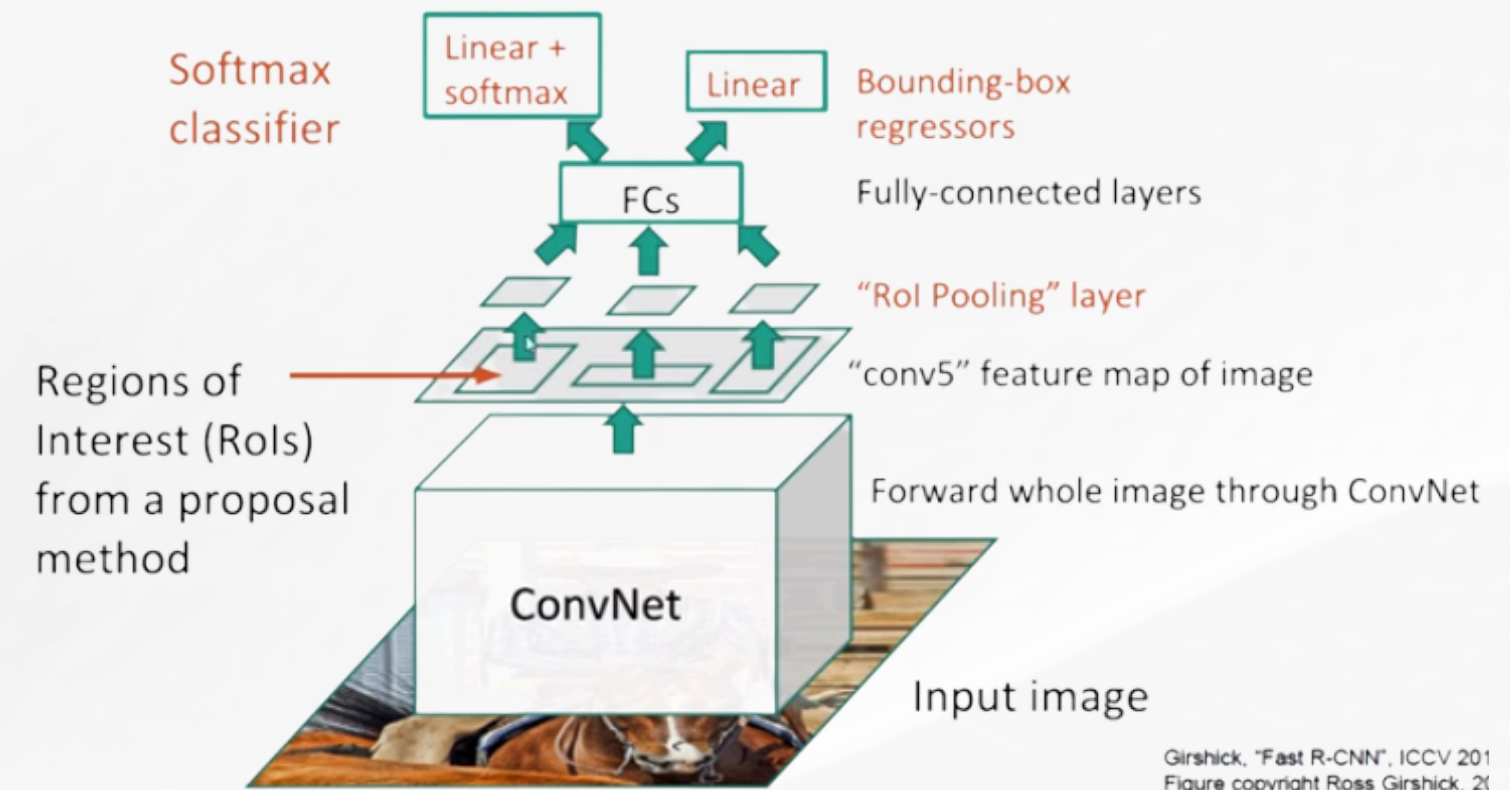
- Region Proposal(Selective Search)
 - 이미지에서 ~수천 개 후보영역(RoI) 생성(IoU/NMS로 중복 억제)
- RoI 정규화(전처리)
 - 각 RoI를 고정 크기(예: 227×227)로 워핑/리사이즈 → CNN 입력 맞춤
- CNN 특징 추출
 - 사전학습 CNN(AlexNet/VGG 등)으로 고정 길이 feature(벡터) 산출
- 분류(SVM, one-vs-rest)
 - 클래스마다 선형 SVM으로 객체/배경 판별
- 박스 회귀(BBox Regressor)
 - 선형 회귀로 (tx, ty, tw, th) 보정 → 위치 정밀화
- 후처리
 - 클래스별 NMS로 최종 박스 선택



Fast R-CNN 아키텍처



Fast R-CNN



전체 파이프라인

1. 단 한 번의 ConvNet 추출(공유계산)

- 입력 이미지를 한 번만 백본(CNN)에 통과 → 공유 특징맵 생성

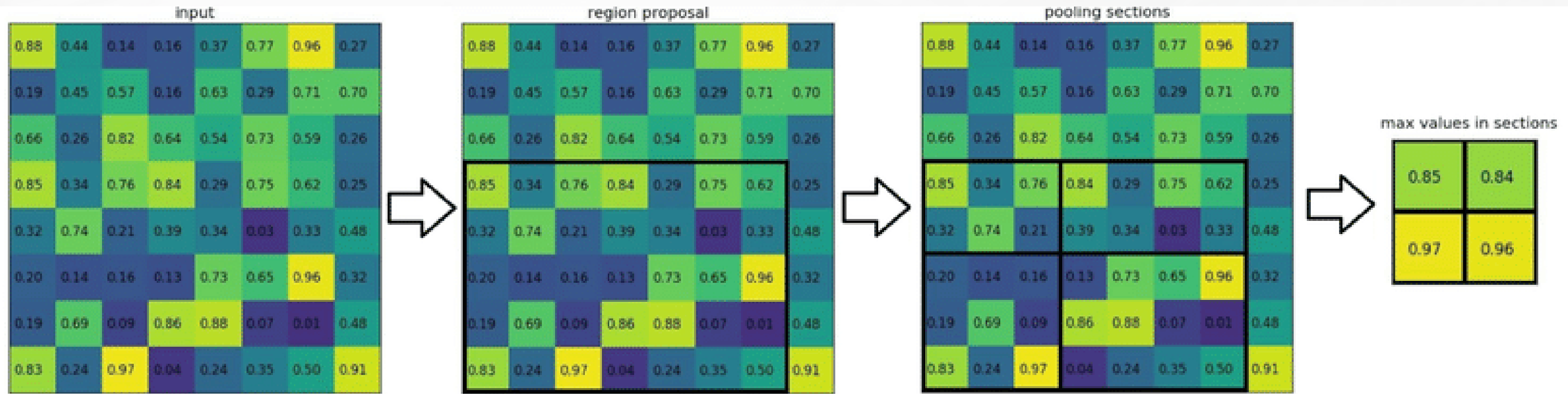
2. RoI Pooling (고정 크기 특성)

- Selective Search 등으로 얻은 RoI를 특징맵 위에서 잘라 RoI Pooling(예: 7×7)으로 고정 크기 특징으로 변환

3. 분류 + 박스 회귀 (멀티태스크 손실)

- FC 헤드 → Softmax 분류와 BBox 회귀를 동시에 학습
- 후처리로 NMS 적용하여 최종 박스 산출

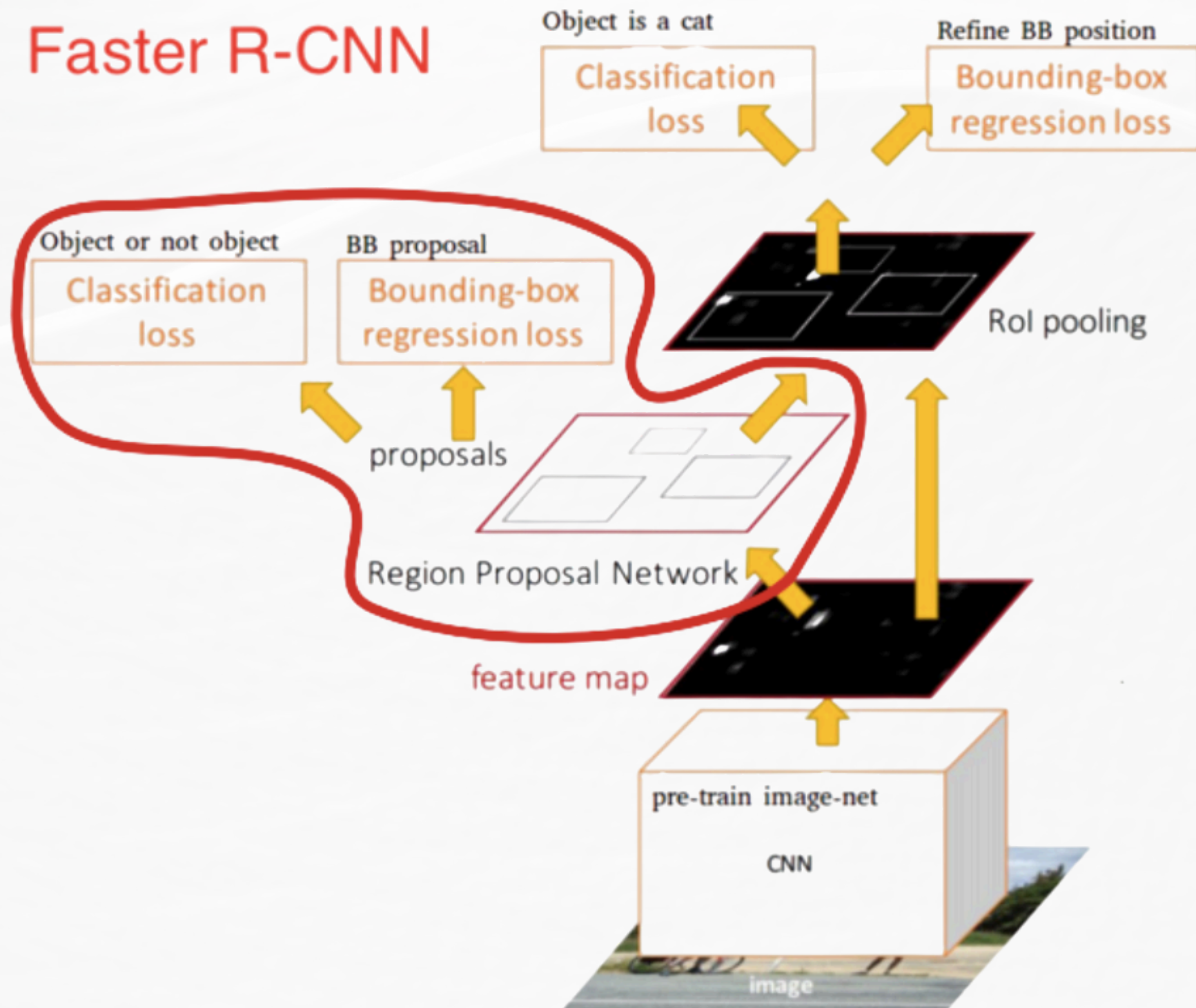
Roi Pooling



- 무엇: 서로 다른 크기의 Roi를 고정 크기(예: 7×7) 특징으로 바꾸는 연산
- 어떻게: Roi를 정수 그리드로 등분(라운딩) → 각 bin마다 max-pooling
- 장점: 빠르고 구현 쉬움

Faster R-CNN 아키텍처

Faster R-CNN



전체 파이프라인

1. Backbone CNN

- 입력 이미지를 한 번 통과 → 공유 feature map 생성

2. RPN(Region Proposal Network)

- feature map의 각 위치에서 anchor별로
 - objectness(물체/배경)
 - bbox 오프셋을 예측 → 상위 proposals 선택(NMS)

3. RoI Pooling/Align

- proposals를 feature map 위에서 잘라 고정 크기 특징으로 변환
- (정밀도가 중요하면 RoI Align 사용)

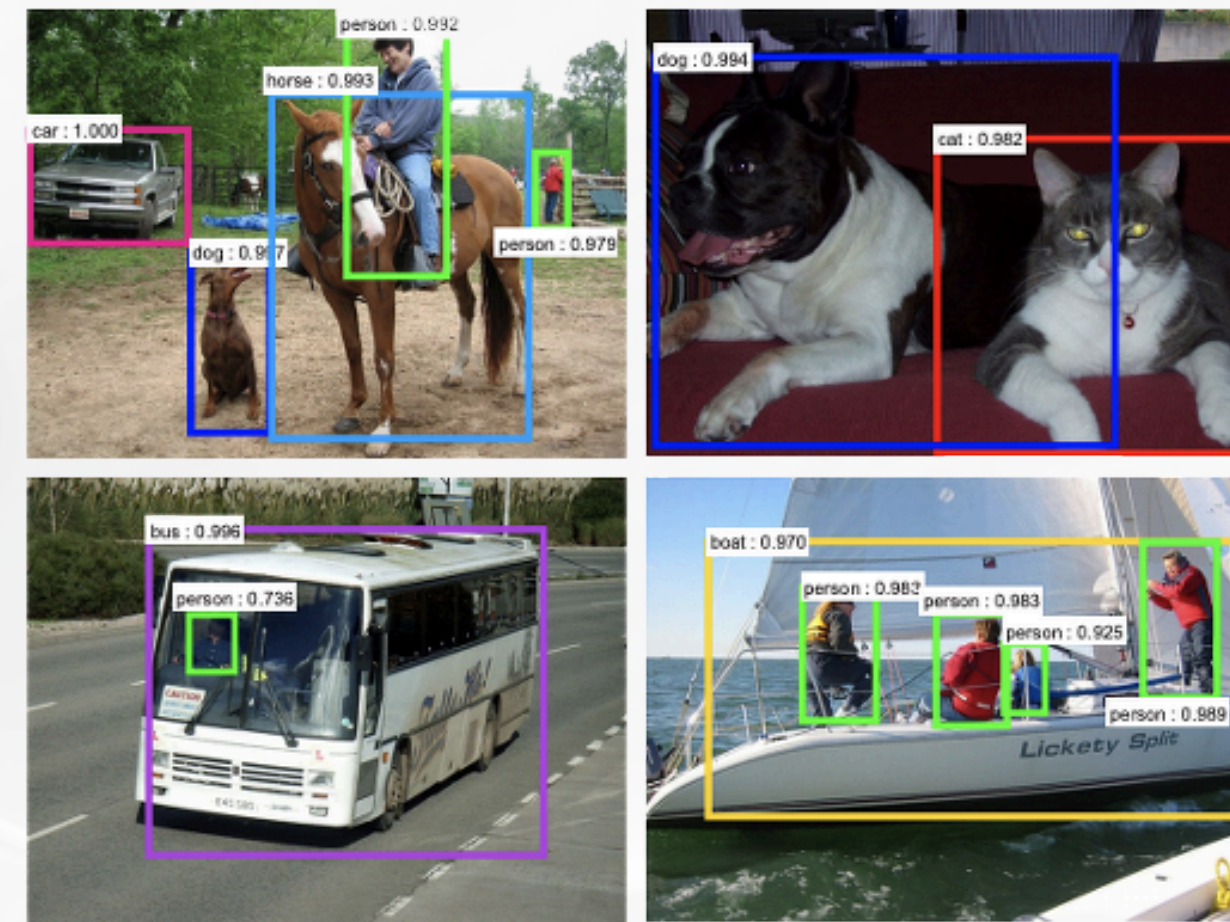
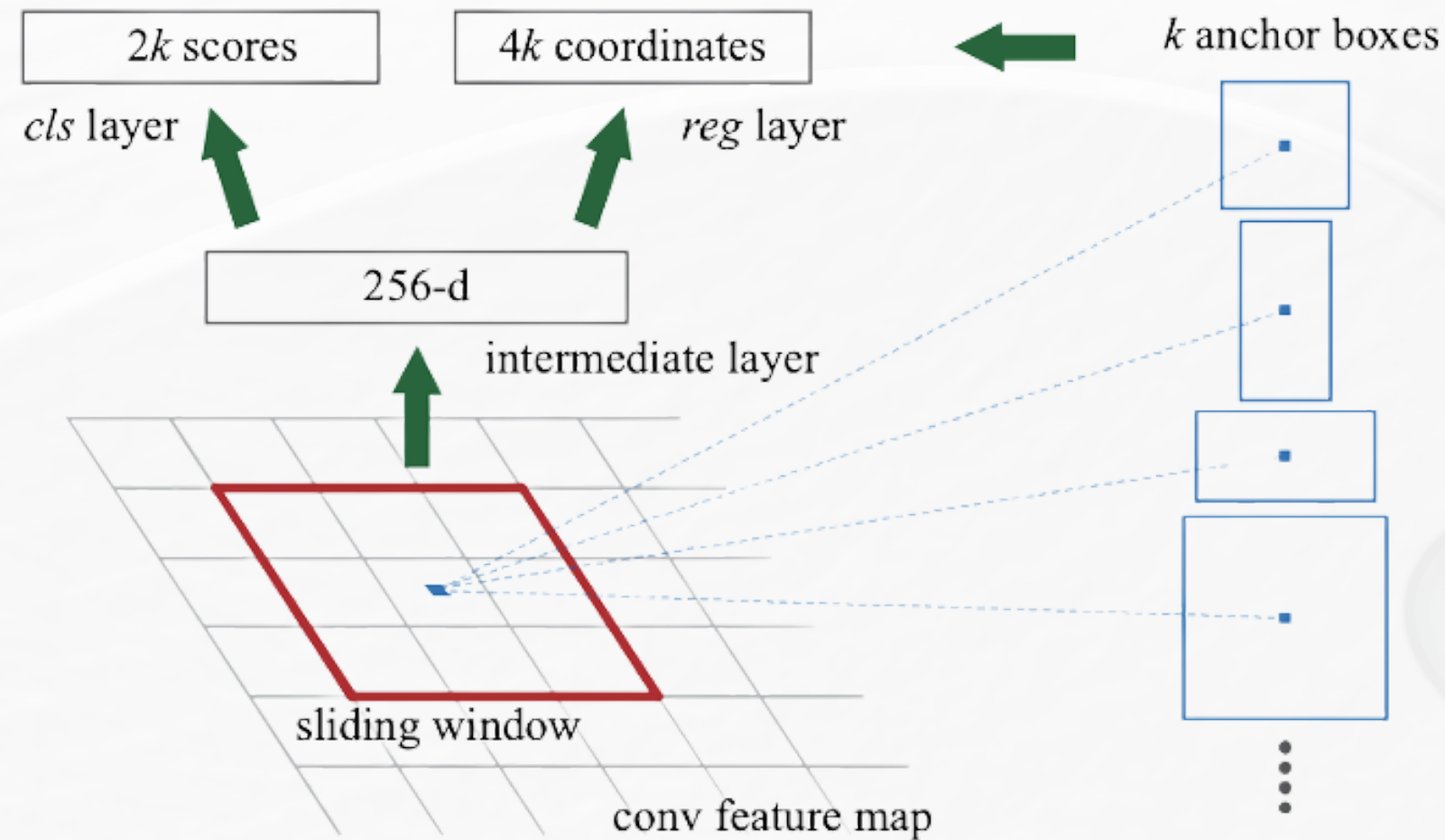
4. Detection Head

- FC/Conv 헤드에서 클래스 분류 + BBox 회귀를 동시에 수행

5. 후처리(NMS)

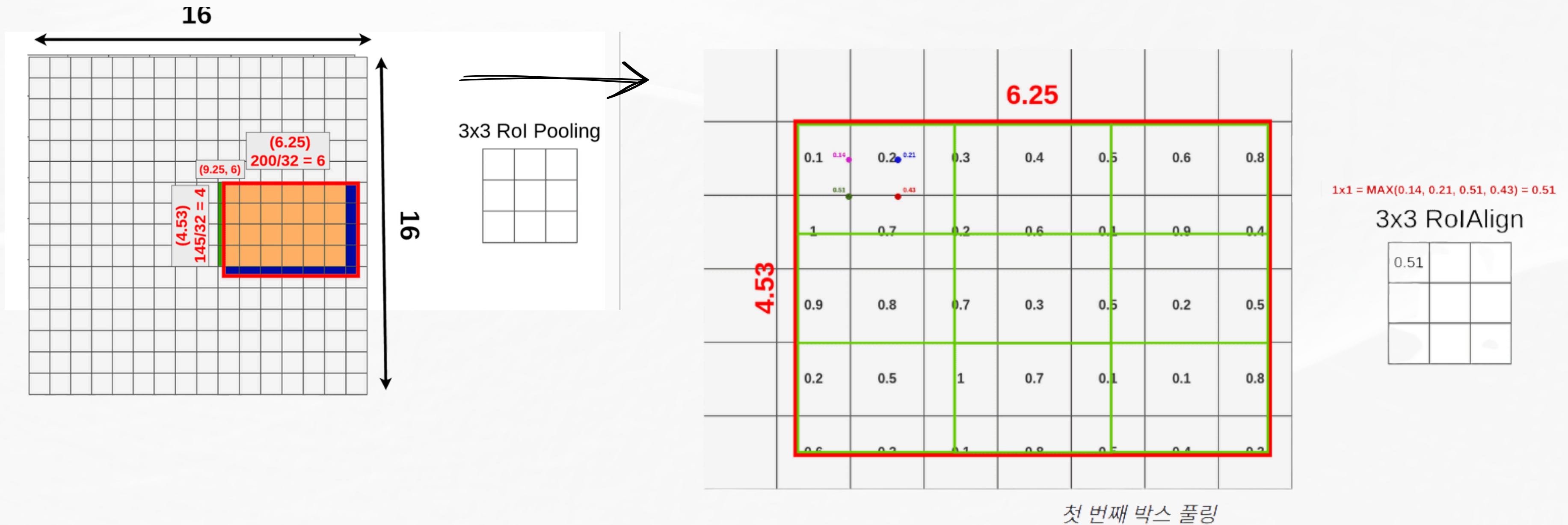
- 클래스별로 겹침 억제 → 최종 (x, y, w, h, class, score) 출력

RPN (Region Proposal Network)



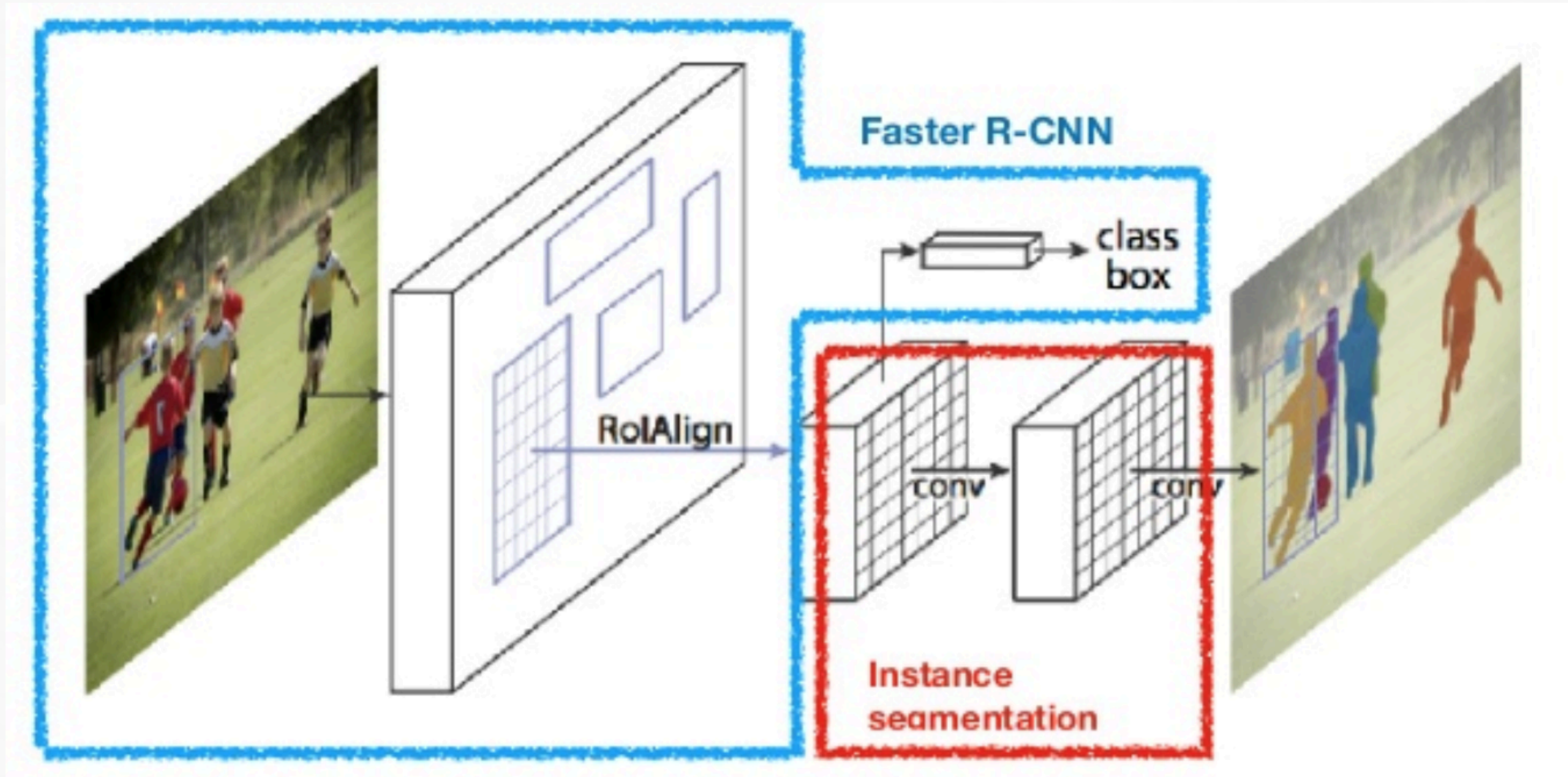
RPN은 Region Proposal Network(영역 제안 네트워크)의 약자
 이는 객체 탐지 모델인 **Faster R-CNN**의 핵심 구성 요소로, 이미지에서 객체가 있을 만한 위치를 효율적으로 찾아내는 역할

Roi Align



- 무엇: ROI를 고정 크기로 바뀌되 좌표를 실수(부동소수)로 정확히 처리
- 어떻게: 라운딩 없이 각 샘플 지점의 값을 bilinear interpolation으로 읽어와 평균/최댓값 집계
- 장점: 정합성↑(픽셀·특징 맵 정렬), 박스/마스크 정확도↑

Mask R-CNN 아키텍처



Faster R-CNN에 마스크 분기(mask head)와 RoI Align을 추가해, 검출(클래스+박스)과 인스턴스 분할(마스크)을 동시에 수행하는 2-Stage 모델

PennFudanPed & 구현 설정

PennFudanPed

- 목적: 보행자(사람) 탐지/분할 실습에 적합한 소규모 데이터
- 구성: 도심 거리 장면의 이미지 + person 박스/마스크 어노테이션
- 사용한 이유: 크기가 작아 모델/파이프라인 비교 실험(R-CNN 계열)과 시각화에 최적

구현 설정

.....



Results | Implementation : RCNN

```
# -----  
# Model  
# -----  
class RCNNHead(nn.Module):  
    def __init__(self, num_classes=2, feat_dim=2048):  
        super().__init__()  
        self.backbone = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)  
        self.backbone.fc = nn.Identity()  
        self.cls_head = nn.Sequential(  
            nn.Linear(feat_dim, 1024), nn.ReLU(inplace=True),  
            nn.Dropout(0.5),  
            nn.Linear(1024, num_classes)  
        )  
        self.reg_head = nn.Sequential(  
            nn.Linear(feat_dim, 1024), nn.ReLU(inplace=True),  
            nn.Linear(1024, 4)  
        )  
  
    def forward(self, x):  
        feat = self.backbone(x)          # [N, 2048]  
        logits = self.cls_head(feat)     # [N, 2]  
        deltas = self.reg_head(feat)     # [N, 4]  
        return logits, deltas
```

Results | Implementation : Fast RCNN

```
# -----  
# Model  
# -----  
class FastRCNNHead(nn.Module):  
    def __init__(self, num_classes=2, roi_out_size=14, feat_dim=2048):  
        super().__init__()  
        backbone = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)  
        self.stem = nn.Sequential(  
            backbone.conv1, backbone.bn1, backbone.relu,  
            backbone.maxpool,  
            backbone.layer1, backbone.layer2, backbone.layer3, backbone.layer4  
        )  
        fc_in = feat_dim * roi_out_size * roi_out_size  
        self.head = nn.Sequential(  
            nn.Linear(fc_in, 1024), nn.ReLU(inplace=True),  
            nn.Dropout(0.5),  
            nn.Linear(1024, 1024), nn.ReLU(inplace=True)  
        )  
        self.cls = nn.Linear(1024, 2)  
        self.reg = nn.Linear(1024, 4)  
  
        nn.init.normal_(self.reg.weight, mean=0.0, std=0.001)  
        nn.init.constant_(self.reg.bias, 0.0)  
        with torch.no_grad():  
            b = self.cls.bias
```

Results | Implementation : Faster RCNN

```
# -----  
# Model  
# -----  
def build_model(num_classes=2):  
    # COCO 사전학습 가중치  
    weights = FasterRCNN_ResNet50_FPN_Weights.COCO_V1  
    model = fasterrcnn_resnet50_fpn(  
        weights=weights,  
        box_detections_per_img=Cfg.max_dets,  
        min_size=Cfg.min_size,    # 입력 정책 고정  
        max_size=Cfg.max_size  
    )  
    # 분류/회귀 헤드 교체(배경+person=2)  
    in_features = model.roi_heads.box_predictor.cls_score.in_features  
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)  
  
    # 내부 임계값/상한 고정(공정 비교)  
    model.roi_heads.score_thresh = Cfg.score_thresh  
    model.roi_heads.nms_thresh = Cfg.nms_iou  
    model.roi_heads.detections_per_img = Cfg.max_dets  
  
    # RPN 제안 상한(과잉 중복 억제: 동일 정책 하에서 안정화)  
    model.rpn.post_nms_top_n_test = 100  
    model.rpn.post_nms_top_n_train = 200
```


Results | Implementation : Mask RCNN

```
# -----  
# Model  
# -----  
def build_model(num_classes=2):  
    model = maskrcnn_resnet50_fpn(weights="DEFAULT",  
                                   |   |   |   |   |   |   |   |   |   |   |   |   |   |  
                                   in_size=Cfg.min_size, max_size=Cfg.max_size)  
    # box predictor 교체  
    in_features = model.roi_heads.box_predictor.cls_score.in_features  
    from torchvision.models.detection.faster_rcnn import FastRCNNPredictor  
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features,  
                                                       |   |   |   |   |   |   |   |   |   |   |   |   |   |  
                                                       num_classes)  
    # mask predictor 교체  
    in_features_mask = model.roi_heads.mask_predictor.conv5_mask.in_channels  
    hidden_layer = 256  
    model.roi_heads.mask_predictor = MaskRCNNPredictor(in_features_mask,  
                                                         |   |   |   |   |   |   |   |   |   |   |   |   |   |  
                                                         hidden_layer, num_classes)  
  
    # 내부 임계값/상한(공정 비교)  
    model.roi_heads.score_thresh = Cfg.score_thresh  
    model.roi_heads.nms_thresh   = Cfg.nms_iou  
    model.roi_heads.detections_per_img = Cfg.max_dets  
  
    # RPN 제안 상한 (Faster와 동일 정책)  
    model.rpn.post_nms_top_n_test  = 100  
    model.rpn.post_nms_top_n_train = 200  
    return model
```

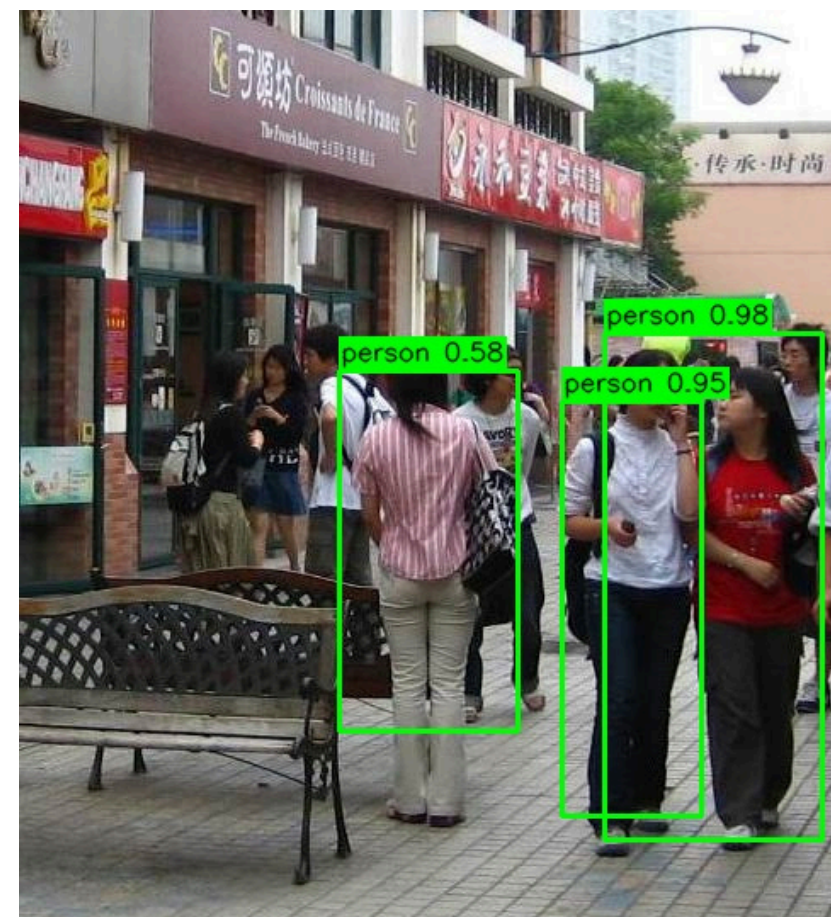
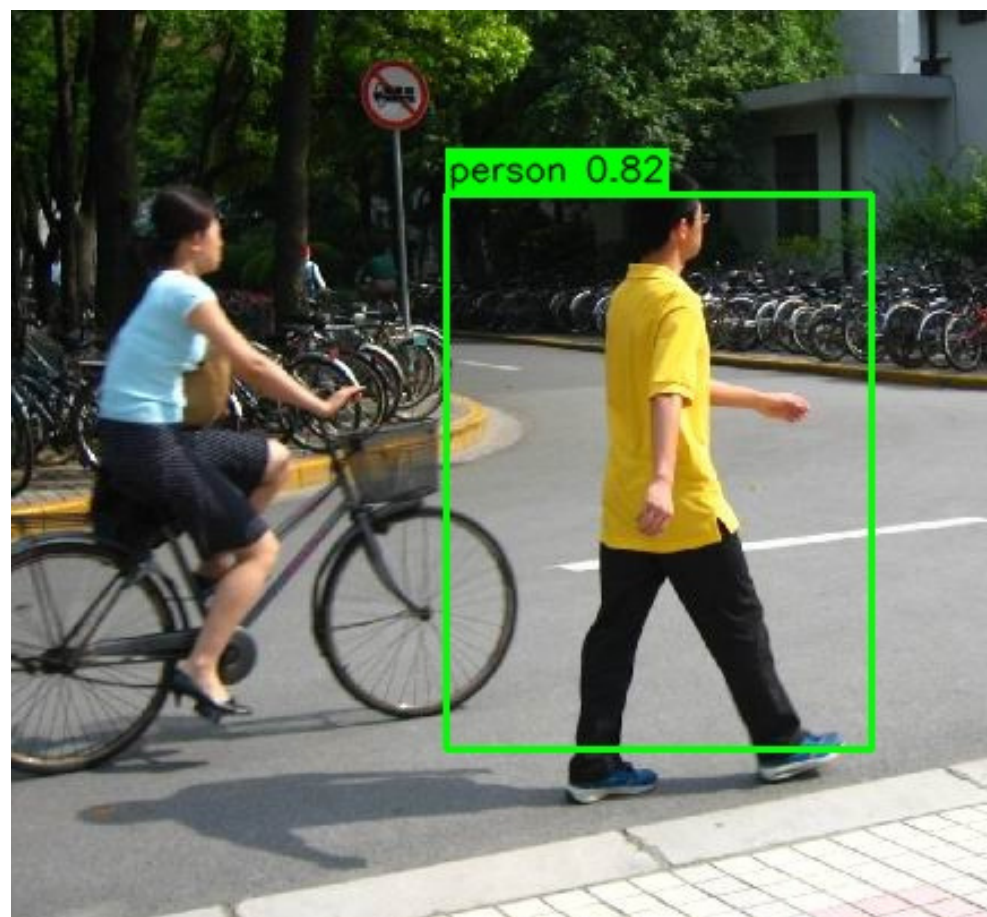

Results | Sample_img : RCNN & Fast

18/21페이지



RCNN

Time = 58.201min
Latency = 5903.4 ms/img
AP@0.5 = 0.179
FPS $\approx$ 0.17

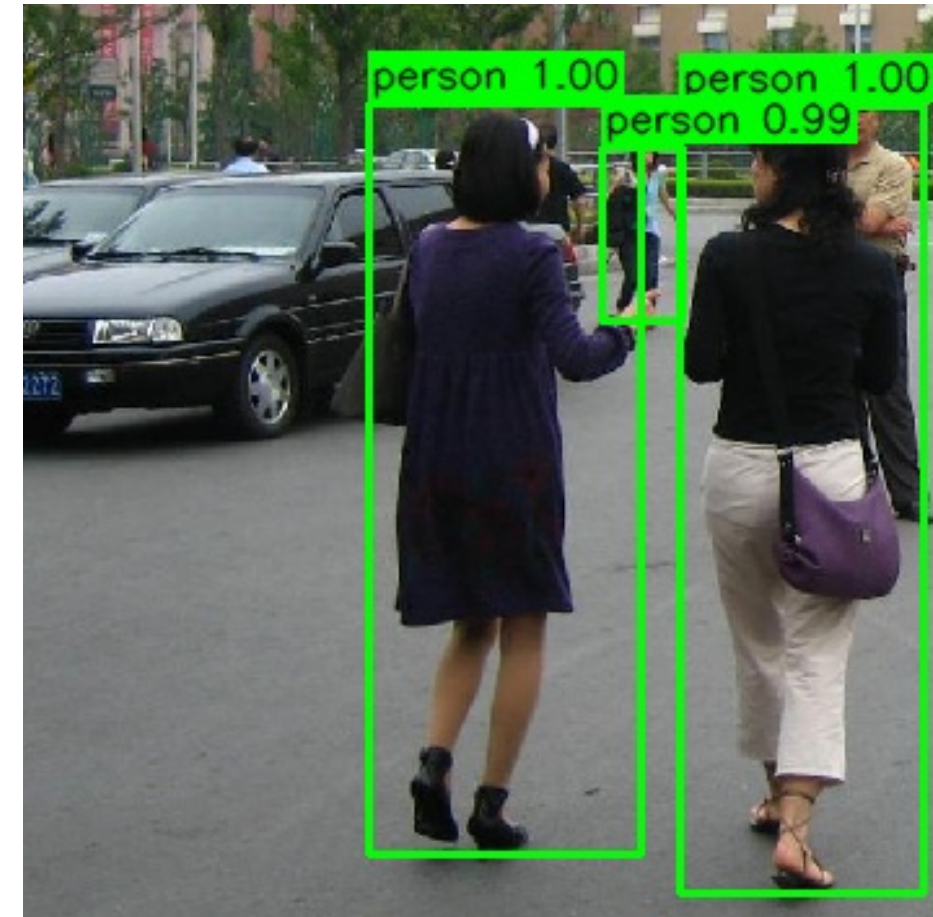
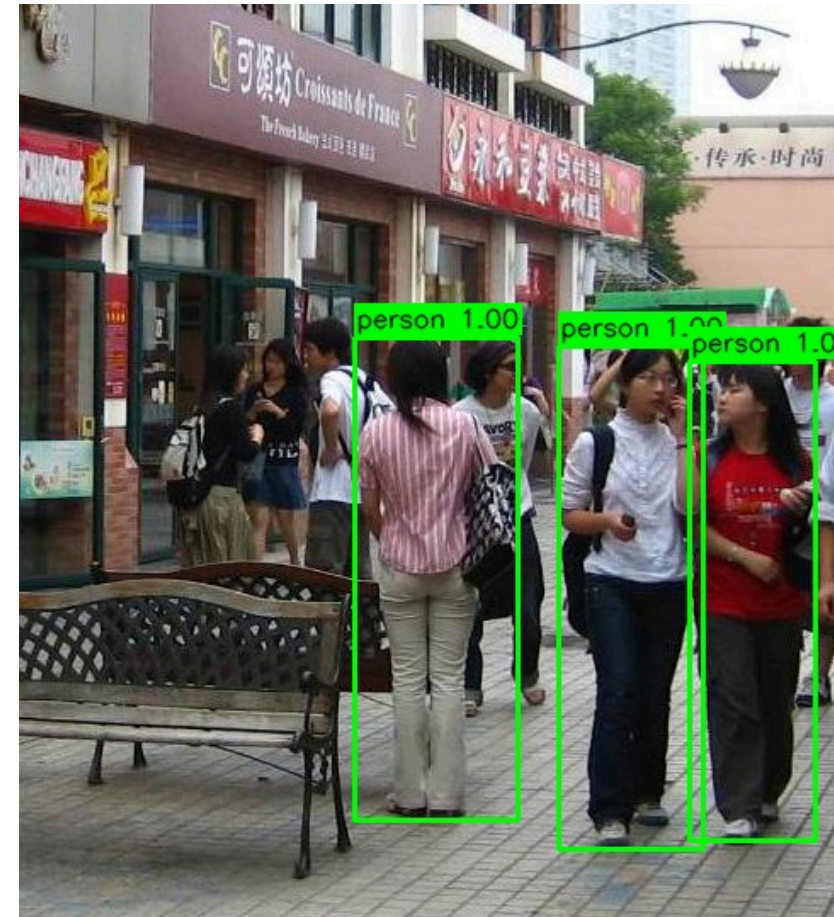
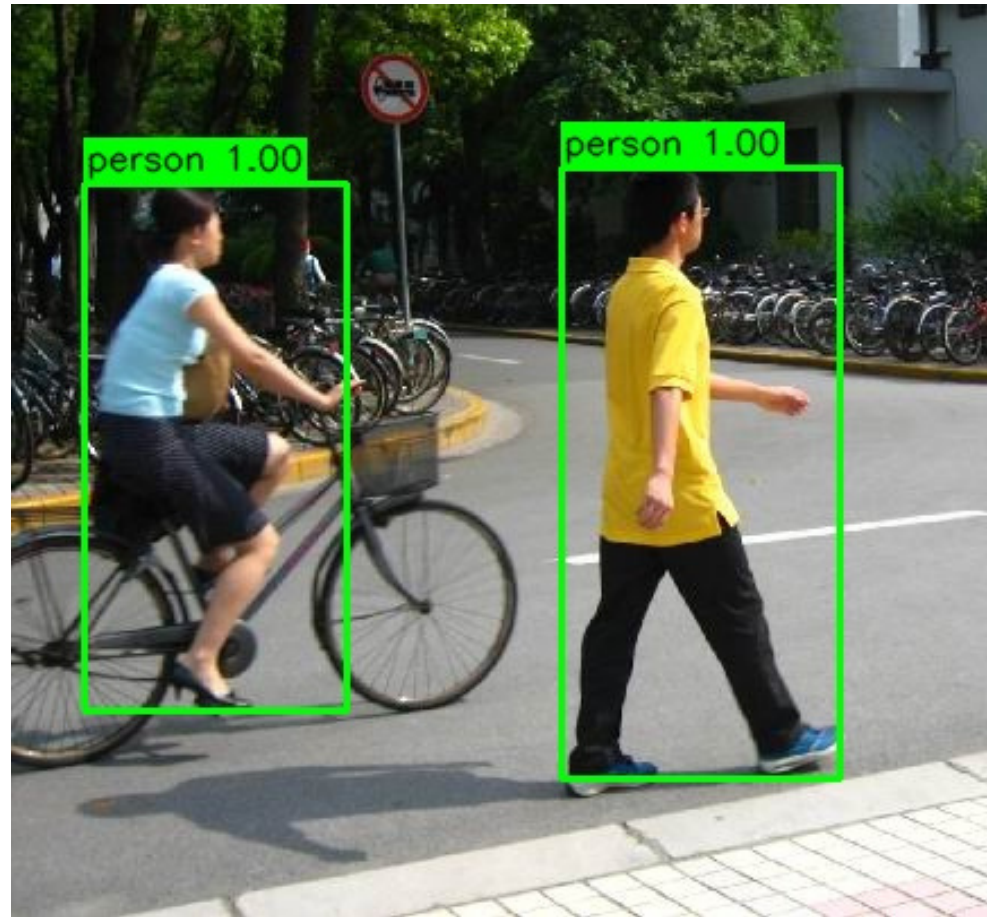


Fast RCNN

Time = 4.434min
Latency = 53.8 ms/img
AP@0.5 = 0.302
FPS $\approx$ 18.60

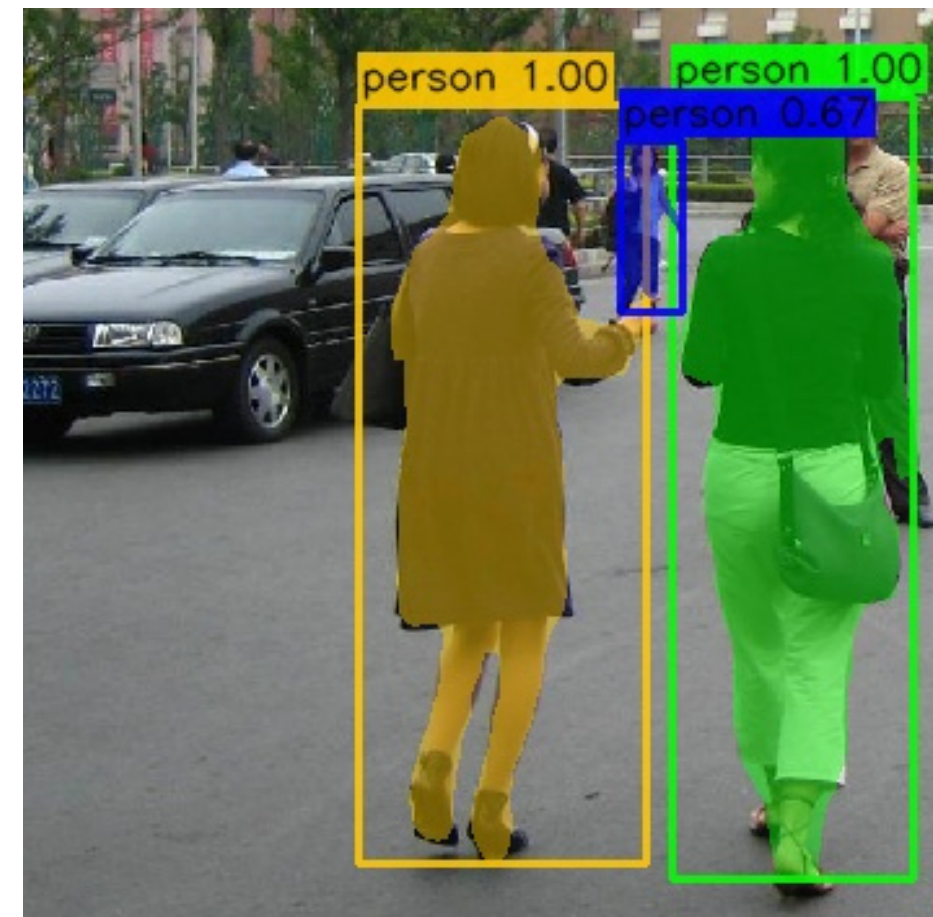
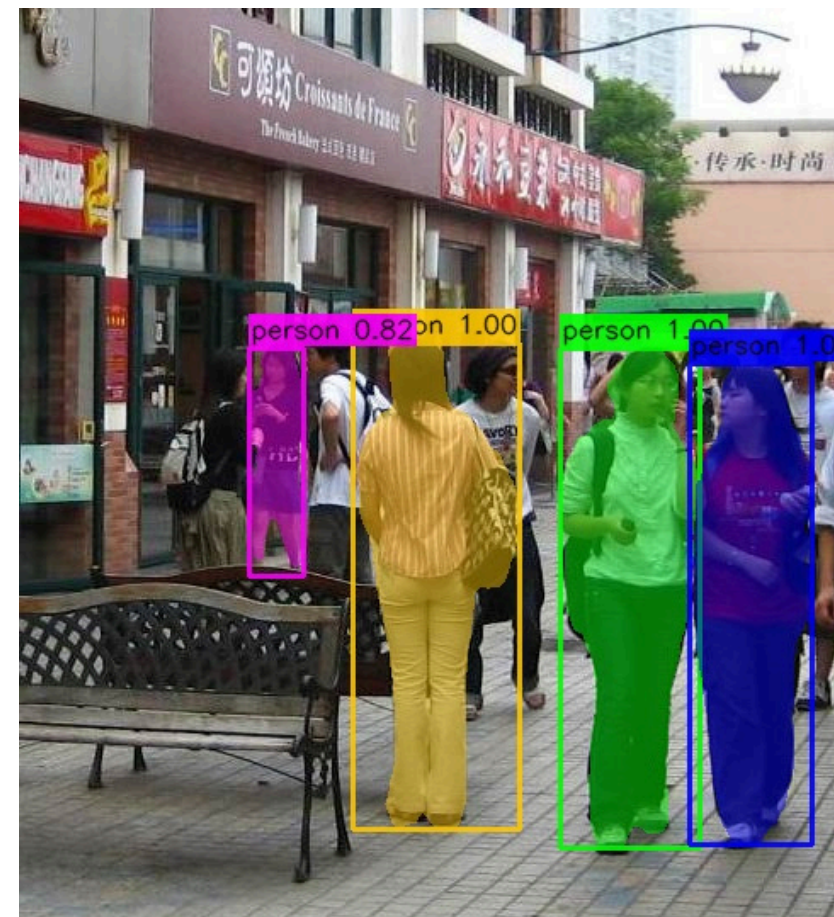
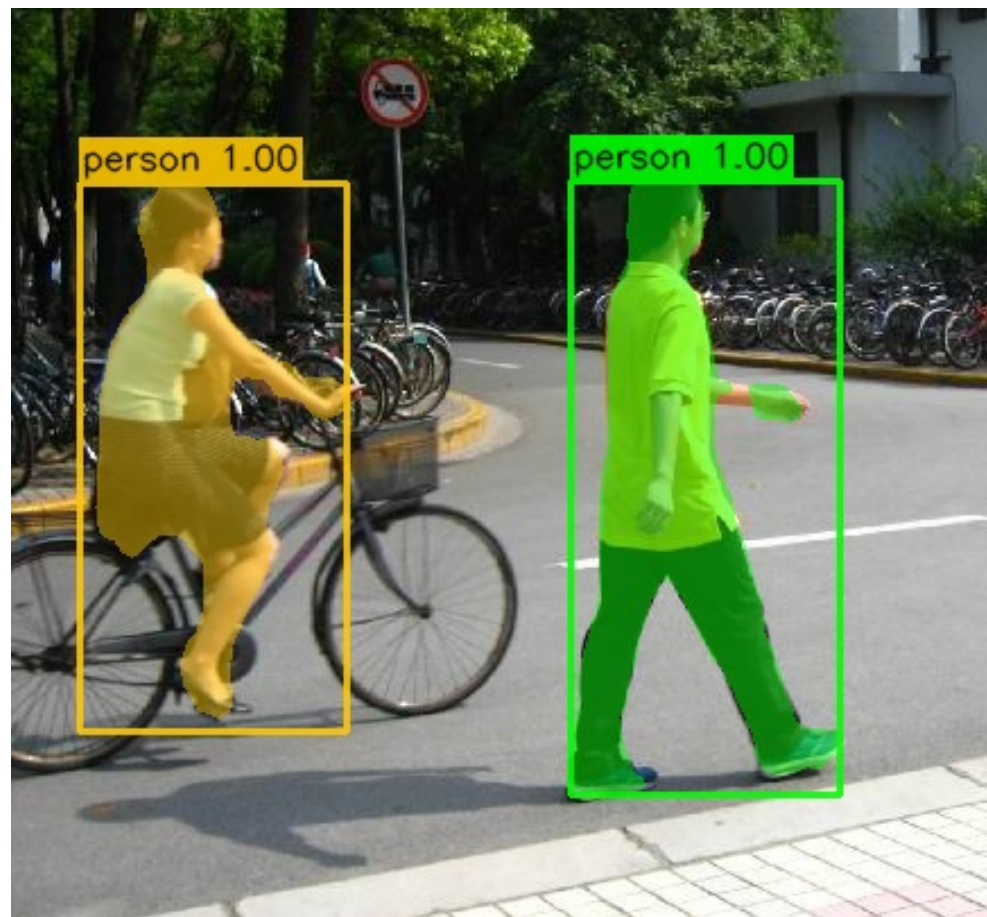
Results | Sample_img : Faster & Mask

19/21페이지



**Faster
RCNN**

**Time = 3.03min
Latency = 0.6 ms/img
AP@0.5 = 0.960
FPS $\approx$ 1784.30**



Mask RCNN

**Time = 3.23min
Latency = 36.7 ms/img
AP@0.5 = 0.957
FPS $\approx$ 27.24**

모델	총 소요 시간	지연(Latency, ms/img)	AP@0.5	FPS
R-CNN	58.201 min	5903.4	0.179	0.17
Fast R-CNN	4.434 min	53.8	0.302	18.60
Faster R-CNN	3.03 min	0.6	0.960	≈1784.30
Mask R-CNN	3.23 min	36.7	0.957	27.24

- R-CNN → Fast: 특징 공유 + RoI Pooling 덕에 지연 급감(5903→54 ms), 정확도 소폭↑
- Fast → Faster: RPN 통합으로 정확도 대폭↑(AP 0.96), 지연 극소화
- Faster ↔ Mask: 박스 성능은 유사, Mask R-CNN은 인스턴스 마스크까지 제공(정밀 컷아웃/편집에 유리)

THANK YOU

FOR COMING